VLSI DESIGN – ASIC FLOW USING OPEN SOURCE TOOL

Complete OpenLane RTL-to-GDSII Workflow Manual

From Windows to Silicon: WSL2, Docker, GHDL, OpenLane, Sky130 PDK, KLayout, and Git/GitHub

Prepared by
Dr. P Rajasekar, Professor, Department of ECE
Sri Krishna College of Engineering and Technology
rajasekarkpr@gmail.com

July 16, 2026

Contents

1 Introduction

This manual is a complete, self-contained guide for taking a digital design from an RTL description (VHDL or Verilog) all the way to a manufacturable chip layout (GDSII), using only free, open-source tools. It consolidates, in one reference document, the full practical workflow: setting up a Linux environment on a Windows machine, installing every required tool, converting VHDL to Verilog, running the OpenLane ASIC implementation flow, interpreting its reports, viewing the resulting layout, and publishing the complete project to GitHub for reproducibility.

Definition

ASIC (Application-Specific Integrated Circuit): a chip designed and fabricated for one specific function, as opposed to a general-purpose, reconfigurable device such as an FPGA. Going from RTL to a fabricable ASIC layout is called the *RTL-to-GDSII flow*, and *GDSII* is the standard binary file format used to describe the final chip layout for fabrication.

Key Idea

The entire toolchain used in this manual – WSL2, Docker, GHDL, OpenLane (Yosys + OpenROAD + Magic + Netgen), the SkyWater Sky130 PDK, and KLayout – is free and open source. No paid EDA license or NDA-restricted process design kit is required to complete every step in this manual.

2 Setting Up the Linux Environment on Windows

OpenLane and its supporting tools run on Linux. On a Windows PC, the standard way to get a full Linux environment is the **Windows Subsystem for Linux, version 2 (WSL2)**, which runs a real Ubuntu kernel alongside Windows with near-native performance.

2.1 Installing WSL2

```
# Run in PowerShell (as Administrator)
wsl --install
```

This installs WSL2 with Ubuntu as the default distribution and prompts for a reboot. After reboot, Ubuntu launches automatically and asks you to create a Linux username and password – this can (and usually should) be different from your Windows username.

Note

Your Windows username and your Linux (WSL2) username are *independent*. If Windows shows `C:\Users\Manivannan`, your Linux prompt might still read `rajasekar1@Rajasekar:~$` – these are two separate identities on two separate filesystems that happen to share one physical machine.

2.2 Verifying and Managing WSL2

```
# From PowerShell -- lists installed distros and their WSL version
wsl -l -v

# Convert a distro to WSL2 if it shows version 1
wsl --set-version Ubuntu 2

# Fully restart the WSL2 subsystem (useful after Docker/GPU issues)
wsl --shutdown
```

2.3 Basic Linux Commands

Command	Purpose
`pwd`	Print current working directory
`ls`, `ls -la`	List files (`-la`: all files, long format)
`cd <dir>`	Change directory
`mkdir -p <path>`	Create a directory (`-p`: create parent dirs as needed)
`cp <src> <dst>`	Copy a file
`cp -r <src> <dst>`	Copy a directory recursively
`mv <src> <dst>`	Move/rename a file or directory
`rm <file>`	Delete a file
`rm -rf <dir>`	Delete a directory and its contents (irreversible – use with care)
`cat <file>`	Print a file's full contents
`grep "text" file`	Search for a pattern inside a file
`find <path> -iname "*.v"`	Search for files by name pattern
`whoami`	Print the current Linux username
`sudo <cmd>`	Run a command with administrator privileges
`chmod +x <file>`	Make a file executable
`export VAR=value`	Set an environment variable for the current session

3 Installing the Open-Source Toolchain

3.1 Docker Desktop and WSL2 Integration

OpenLane ships as a Docker container, so Docker Desktop must be installed on Windows with WSL2 integration enabled.

1. Install Docker Desktop for Windows from the official site.
2. Open Docker Desktop → **Settings** → **Resources** → **WSL Integration**.

3. Enable *"Enable integration with my default WSL distro"* **and** toggle ON your specific distro (e.g. Ubuntu) individually – both switches are required.

4. Click **Apply & Restart**.

```
# Verify Docker is reachable from inside Ubuntu
docker version
docker info
```

3.1.1 Fixing the Docker Credential Error

A very common first-time error on Windows + WSL2 is:

```
error getting credentials - err: exit status 1
```

Key Insight / Design Implication

This happens because Docker tries to use the Windows Credential Manager from inside Linux, which does not work reliably across the WSL2 boundary. The fix is to reset Docker's config file to use no external credential helper.

```
mkdir -p ~/.docker
cat > ~/.docker/config.json << 'EOF'
{
  "auths": {}
}
EOF

# Verify:
docker pull hello-world
docker run hello-world
```

3.2 GHDL (VHDL Simulation and Synthesis)

```
sudo apt update
sudo apt install -y ghdl
```

Note

Install GHDL *natively inside* Ubuntu/WSL2. A Windows-only GHDL installation is invisible from inside the Linux terminal and inside the Docker container.

3.3 KLayout (Layout Viewer)

```
sudo apt install -y klayout
```

WSL2's built-in `WSLg` feature lets KLayout's graphical window open directly on the Windows desktop with no extra configuration.

3.4 Git

```
git --version # check if already installed
sudo apt install -y git # install if missing
```

3.5 Icarus Verilog (optional, for quick simulation)

```
sudo apt install -y iverilog
```

4 Transferring Files Between Windows and Linux

WSL2 mounts every Windows drive under `/mnt/`.

Windows path	Path inside Ubuntu (WSL2)
C:\Users\YourName\	/mnt/c/Users/YourName/
E:\ASIC_Work	/mnt/e/ASIC_Work

4.1 Windows → Linux

```
mkdir -p ~/ASIC_Lab/projects/MyDesign/src
cp /mnt/e/ASIC_Work/*.vhd ~/ASIC_Lab/projects/MyDesign/src/
```

4.2 Linux → Windows

```
mkdir -p /mnt/e/ASIC_Work
cp ~/my_screenshot.png /mnt/e/ASIC_Work/
```

Key Idea

You can also open the current Linux folder directly in Windows File Explorer with `explorer.exe .`, which is handy for drag-and-drop transfers.

Key Insight / Design Implication

Always do your actual work *inside* the Linux filesystem (`~/ASIC_Lab/...`), copying files in and out of `/mnt/c` or `/mnt/e` only at the start/end. Tool performance and file-permission handling are significantly better on the native Linux filesystem than when operating directly on a mounted Windows drive.

5 VHDL to Verilog Conversion

OpenLane's synthesis stage (Yosys) consumes Verilog. If your design is in VHDL, convert it first using GHDL's synthesis backend.

5.1 Single-File Designs

```
ghdl -a MyDesign.vhd
ghdl synth --out=verilog MyDesign > MyDesign.v
```

5.2 Multi-File Designs (Gate Libraries)

Real designs are usually split across multiple VHDL files: low-level gate primitives plus a top-level file that instantiates them. GHDL must analyze files in *dependency order* – leaf components first, top-level last.

Finding the Dependency Order

```
# Files that instantiate other components (analyze these LAST)
grep -l "component\|port map" *.vhd

# Entity name declared in each file
grep -i "^entity" *.vhd

# What each file actually instantiates
grep -B2 -A1 "port map" *.vhd
```

```
# Example: leaf gates first
ghdl -a feynman_gate.vhd
ghdl -a hng_gate.vhd
ghdl -a bvf_gate.vhd

# Then intermediate blocks that use the leaf gates
ghdl -a gf_mult02.vhd
ghdl -a gf_mult04.vhd

# Top-level entity last
ghdl -a MyDesign.vhd

# Synthesize to Verilog (use the ENTITY name, not the filename)
ghdl synth --out=verilog TopEntityName > MyDesign.v
```

Key Insight / Design Implication

If a file errors with "entity X not found," it was analyzed too early. Move it later in the sequence and rerun `ghdl -a` from that point.

5.3 Sanity-Checking the Converted Netlist

```
cat MyDesign.v
grep -i module MyDesign.v
```

Confirm the top-level module name and check whether it has a `clk` port – this determines how the OpenLane `config.json` must be written (Section 6.3).

6 The OpenLane RTL-to-GDSII Flow

Definition

OpenLane is an automated, open-source digital ASIC implementation flow. It chains together several independently developed open-source tools into one pipeline that takes a synthesizable Verilog netlist all the way to a signed-off GDSII layout.

6.1 The Tools Inside OpenLane and Why Each Matters

Tool	Stage	Importance
Yosys	Logic synthesis	Converts the RTL Verilog netlist into a gate-level netlist mapped onto the target standard cell library.
OpenROAD	Floorplan, placement, CTS, routing, STA	The core place-and-route engine; decides die/core size, positions every standard cell, builds the clock tree (if any), and routes all metal interconnect.
Magic	DRC, GDSII streaming	A VLSI layout tool used for Design Rule Checking and generating the final GDSII file.
Netgen	LVS	Performs Layout-versus-Schematic verification, confirming the physical layout is electrically identical to the synthesized netlist.
KLayout	DRC (cross-check), layout viewing	A second, independent layout viewer/DRC engine used to cross-verify Magic's results and to visually inspect the design.

6.2 Project Directory Structure

```
mkdir -p ~/ASIC_Lab/projects/MyDesign/src
cp MyDesign.v ~/ASIC_Lab/projects/MyDesign/src/
```

Required structure:

```
MyDesign/
  config.json
  src/
    MyDesign.v
```

6.3 Writing config.json

Combinational design (no clk port)

```
cat > ~/ASIC_Lab/projects/MyDesign/config.json << 'EOF'
{
    "DESIGN_NAME": "MyDesign",
    "VERILOG_FILES": "dir::src/MyDesign.v",
    "CLOCK_PERIOD": 10,
    "FP_CORE_UTIL": 40,
    "PL_TARGET_DENSITY": 0.45,
    "DIODE_INSERTION_STRATEGY": 4,
    "RUN_CTS": 0,
    "PDK": "sky130A",
    "STD_CELL_LIBRARY": "sky130_fd_sc_hd"
}
EOF
```

Clocked (sequential) design

```
cat > ~/ASIC_Lab/projects/MyDesign/config.json << 'EOF'
{
    "DESIGN_NAME": "MyDesign",
    "VERILOG_FILES": "dir::src/MyDesign.v",
    "CLOCK_PORT": "clk",
    "CLOCK_PERIOD": 10,
    "FP_CORE_UTIL": 40,
    "PL_TARGET_DENSITY": 0.45,
    "PDK": "sky130A",
    "STD_CELL_LIBRARY": "sky130_fd_sc_hd"
}
EOF
```

Key Insight / Design Implication

`DESIGN_NAME` must exactly match your top-level Verilog *module* name, not the file-name. If your VHDL entity was named differently from the file (common after GHDL conversion), check with `grep -i module *.v` first.

6.4 Pulling and Launching the OpenLane Container

```
# One-time download (~8-10 GB)
docker pull efabless/openlane:latest
docker images

# Launch, mounting your project folders into the container
docker run -it \
  -v ~/ASIC_Lab/projects:/openlane/designs \
  -v ~/ASIC_Lab/results:/openlane/results \
  efabless/openlane:latest
```

Note

Every `docker run` creates a *brand-new* container. Files inside your mounted `designs`/`results` folders persist, but anything installed *inside* the previous container's own filesystem (such as the PDK) does not carry over – you may need to re-run the PDK setup step below each session, unless you switch to `docker start -ai <container_id>` to resume the same container instead of creating a new one.

6.5 PDK Setup with Volare

```
volare enable --pdk sky130 bdc9412b3e468c102d01b7cf6337be06ec6e9c9a
```

Key Insight / Design Implication

Volare downloads PDK files from GitHub. Unauthenticated requests are capped at 60/hour and can fail with `403 rate limit exceeded`. Fix by exporting a GitHub Personal Access Token first:

```
export GITHUB_TOKEN=ghp_your_token_here
volare enable --pdk sky130 bdc9412b3e468c102d01b7cf6337be06ec6e9c9a
```

6.6 Running the Flow

```
flow.tcl -design /openlane/designs/MyDesign
```

A successful run ends with:

```
[SUCCESS]: Flow complete.
```

6.7 Understanding Each Flow Stage

Stage	Name	What Happens / Why It Matters
1	Synthesis	RTL Verilog → gate-level netlist mapped to the standard cell library (Yosys).

Stage	Name	What Happens / Why It Matters
2	Floorplanning	Defines the die/core boundary, I/O pin locations, and the power distribution network (PDN).
3	Placement	Positions every standard cell inside the core area, first globally then with detailed legalization.
4	CTS	Builds a balanced clock tree for sequential designs (skipped for purely combinational designs).
5	Routing	Draws the actual metal wires connecting every cell, first at a coarse (global) level, then exact (detailed) tracks.
6	STA	Static Timing Analysis – checks setup/hold timing across multiple process corners.
7	DRC	Design Rule Check – verifies the layout obeys the PDK's fabrication geometry rules.
8	LVS	Layout-versus-Schematic – confirms the physical layout is electrically identical to the netlist.
9	GDSII Export	The final, manufacturable layout file is generated.

7 Viewing and Interpreting Results

7.1 Locating a Run and Its Reports

```
ls /openlane/designs/MyDesign/runs/
RUN=<paste_the_run_folder_name_here>
```

7.2 Key Reports

```
# Cell count, gate breakdown, post-synthesis area
cat /openlane/designs/MyDesign/runs/$RUN/reports/synthesis/1-synthesis.AREA_0
    .stat.rpt

# Die and core area
cat /openlane/designs/MyDesign/runs/$RUN/reports/floorplan/3-
    initial_fp_die_area.rpt
cat /openlane/designs/MyDesign/runs/$RUN/reports/floorplan/3-
    initial_fp_core_area.rpt

# Multi-corner power
cat /openlane/designs/MyDesign/runs/$RUN/reports/signoff/31-sta-rcx_min/
    multi_corner_sta.power.rpt
```

```
# Timing summary (WNS, TNS, worst slack)
cat /openlane/designs/MyDesign/runs/$RUN/reports/signoff/31-sta-rcx_min/
    multi_corner_sta.summary.rpt

# DRC and LVS signoff
cat /openlane/designs/MyDesign/runs/$RUN/reports/signoff/drc.rpt
cat /openlane/designs/MyDesign/runs/$RUN/reports/signoff/*.lvs.rpt

# Consolidated metrics (HPWL, wirelength, cell counts, power -- all in one
    CSV)
cat /openlane/designs/MyDesign/runs/$RUN/reports/metrics.csv
```

Key Idea

`COUNT: 0` in `drc.rpt` and "`Total errors = 0`" in the LVS report are the two numbers that confirm a manufacturable, signed-off layout. A completed flow (`[SUCCESS]: Flow complete.`) is *not* by itself proof of a clean layout – always check these two reports explicitly.

7.3 Viewing the Final GDSII Layout

Exit the container first (`exit`), then run KLayout directly on the WSL2 host:

```
klayout ~/ASIC_Lab/projects/MyDesign/runs/$RUN/results/final/gds/MyDesign.gds
```

WSLg opens the KLayout window directly on the Windows desktop.

7.4 Viewing Intermediate Stages (Floorplan / Placement / Routing)

The final GDSII only shows the finished chip. To see the *floorplan*, *placement*, and *routing* stages separately, OpenLane saves a DEF checkpoint after each:

```
find ~/ASIC_Lab/projects/MyDesign/runs/$RUN/results -iname "*.def"
find ~/ASIC_Lab/projects/MyDesign/runs/$RUN/tmp -iname "merged*.lef"
```

Key Insight / Design Implication

KLayout's **File** → **Import** → **DEF Reader** dialog can crash on some builds (a known `liblefdef_ui.so` bug). The reliable workaround is to register the LEF under a custom *Technology* first, then open the DEF file normally:

1. Open KLayout, click the technology icon in the toolbar → **Technology Manager**.
2. Click + to add a new technology (e.g. `sky130_view`).
3. Under **Reader Options** → **LEF/DEF**, add the `merged.nom.lef` file found above.

4. Set this technology as active, then use plain **File** → **Open** on any stage's `.def` file.

The same technology (and LEF) can be reused for every design's DEF files, since the standard cell library is identical across designs on the same PDK – only the DEF file changes.

8 Version Control with Git and GitHub

8.1 Configuring Git (One-Time Setup)

```
git config --global user.email "you@example.com"
git config --global user.name "Your Name"
```

8.2 Creating a GitHub Repository

1. Go to https://github.com/new in a browser.
2. Give it a name, set it to **Public** (so reviewers can access it without an account).
3. Do *not* initialize with a README (you will push your own).
4. Click **Create repository** – note the HTTPS URL shown.

8.3 Recommended Repository Structure

```
project-repo/
  README.md
  LICENSE
  rtl/
    <design>/
      <design>.v
  openlane/
    <design>/
      config.json
  reports/
    <design>/
      synthesis/
      floorplan/
      signoff/
  figures/
    <design>/
```

8.4 Initializing, Committing, and Pushing

```
mkdir -p ~/ASIC_Lab/my-project-repo
cd ~/ASIC_Lab/my-project-repo
git init

# ... copy in rtl/, openlane/, reports/, figures/, README.md, LICENSE ...

git add .
git commit -m "Initial commit: RTL, OpenLane config, reports, figures"
git branch -M main
git remote add origin https://github.com/yourusername/project-repo.git
git push -u origin main
```

Key Insight / Design Implication

When prompted for a password, GitHub no longer accepts your plain account password over HTTPS. Use a **Personal Access Token** instead:

1. https://github.com/settings/tokens → **Generate new token (classic)**.
2. Check the `repo` scope (needed for push access).
3. Generate, copy the token (starts with `ghp_...`), and paste it as the password when Git prompts.

8.5 Uploading Results and Layout Images

```
RUN=RUN_2026.07.11_09.41.23
cp ~/ASIC_Lab/projects/MyDesign/runs/$RUN/reports/synthesis/1-synthesis.
    AREA_0.stat.rpt \
   reports/MyDesign/synthesis/
cp ~/ASIC_Lab/projects/MyDesign/runs/$RUN/reports/signoff/{drc.rpt,*.lvs.rpt,
    metrics.csv} \
   reports/MyDesign/signoff/
cp ~/floorplan_screenshot.png ~/placement_screenshot.png ~/routing_screenshot
    .png \
   figures/MyDesign/

git add .
git commit -m "Add MyDesign synthesis/signoff reports and layout figures"
git push
```

9 Troubleshooting Quick Reference

Symptom	Likely Cause / Fix
`docker: command not found` in Ubuntu	WSL Integration not enabled for your distro in Docker Desktop settings.
`error getting credentials`	Reset `~/.docker/config.json` to `{"auths": {}}`.
`No such file or directory` on `flow.tcl`	Run `find / -iname "flow.tcl" 2>/dev/null` to locate the real path (varies by image build/version).
`No design configuration found`	Pass the full path: `flow.tcl -design /openlane/designs/MyDesign`.
`Standard Cell Library not found in PDK`	PDK not yet downloaded in *this* container – run `volare enable` again (each fresh container needs it once).
`403 rate limit exceeded` (Volare)	GitHub API limit hit – set `GITHUB_TOKEN` or wait 20–30 minutes.
Command works in one terminal but not another	Check `whoami` – you may be logged in as a different Linux user than the one where tools were installed.
KLayout crashes on **File** → **Import** → **DEF**	Use the Technology Manager workaround (Section 7.4) instead of the Import wizard.
Path like `.../floorplan/MyDesign.def` typed at the bash prompt gives "Permission denied"	That path is meant for KLayout's **File** → **Open** dialog, not the terminal – it is a data file, not a program.

10 End-to-End Command Summary

Key Idea

This section is a fast copy-paste reference once each step above is understood.

```
# 1. VHDL to Verilog (skip if already Verilog)
ghdl -a MyDesign.vhd
ghdl synth --out=verilog MyDesign > MyDesign.v

# 2. Project setup
mkdir -p ~/ASIC_Lab/projects/MyDesign/src
cp MyDesign.v ~/ASIC_Lab/projects/MyDesign/src/

# 3. config.json
cat > ~/ASIC_Lab/projects/MyDesign/config.json << 'EOF'
{
    "DESIGN_NAME": "MyDesign",
    "VERILOG_FILES": "dir::src/MyDesign.v",
    "CLOCK_PERIOD": 10,
    "FP_CORE_UTIL": 40,
```

```
    "PL_TARGET_DENSITY": 0.45,
    "RUN_CTS": 0,
    "PDK": "sky130A",
    "STD_CELL_LIBRARY": "sky130_fd_sc_hd"
}
EOF

# 4. Launch OpenLane container
docker run -it \
  -v ~/ASIC_Lab/projects:/openlane/designs \
  -v ~/ASIC_Lab/results:/openlane/results \
  efabless/openlane:latest

# 5. Inside the container: fetch PDK (first time only) and run
volare enable --pdk sky130 bdc9412b3e468c102d01b7cf6337be06ec6e9c9a
flow.tcl -design /openlane/designs/MyDesign

# 6. Check signoff
RUN=<your_run_folder_name>
cat /openlane/designs/MyDesign/runs/$RUN/reports/signoff/drc.rpt
cat /openlane/designs/MyDesign/runs/$RUN/reports/signoff/*.lvs.rpt

# 7. View layout (after exiting the container)
exit
klayout ~/ASIC_Lab/projects/MyDesign/runs/$RUN/results/final/gds/MyDesign.gds

# 8. Publish to GitHub
cd ~/ASIC_Lab/my-project-repo
git add .
git commit -m "Add MyDesign results"
git push
```

Key Insight / Design Implication

Replace every `MyDesign` above with your actual design/module name. A mismatch between `DESIGN_NAME` in `config.json` and the actual top-level Verilog module name is the single most common beginner mistake in this entire flow.